

FactoryMethod

1. Explicación del patrón

Factory Method es un patrón de creación que define una forma de crear objetos, pero deja a las subclases la decisión de qué clase concreta instanciar.

Su objetivo es crear objetos diferentes, aunque de la misma naturaleza, en tiempo de ejecución.

La idea principal es que el cliente o la clase principal no cree directamente los objetos concretos usando sus constructores. En su lugar, se define un método fábrica encargado de crear el producto.

La clase base declara ese método fábrica, y las subclases concretas lo sobrescriben para decidir qué producto concreto se debe crear.

De esta forma, el código que usa los objetos puede trabajar con una interfaz o clase abstracta común, sin depender de las clases concretas.

2. Cuándo se utiliza

Se utiliza el patrón Factory Method cuando:

- No se puede prever de antemano la clase concreta del objeto que se debe crear.
- Una clase quiere que sus subclases especifiquen los objetos que crea.
- Se necesitan crear objetos diferentes, pero todos de la misma naturaleza.
- Se quiere evitar que el código cliente dependa directamente de clases concretas.
- Las clases delegan la responsabilidad de creación en una de varias clases auxiliares.
- Se quiere localizar en qué subclase auxiliar concreta se delega la creación.
- Se quiere separar el código de creación de objetos del código que los utiliza.
- Se quiere poder añadir nuevos tipos de productos sin romper el código cliente existente.
- Hay condicionales que deciden qué clase concreta instanciar y se quiere organizar mejor esa lógica.

3. Cómo aplicarlo

Para aplicar Factory Method se suelen definir estos participantes:

Producto

Es la interfaz o clase base común de todos los objetos que se pueden crear.

Define las operaciones que todos los productos concretos deben cumplir.

El cliente trabaja con este tipo general, no con las clases concretas.

Ejemplo conceptual:

- `Producto`

Ejemplos del material:

- `Conexion`
- `Figura`

Producto concreto

Es una implementación concreta del producto.

Cada producto concreto representa una variante distinta del objeto que se puede crear.

Ejemplos del material:

- `ConexionMySQL`
- `ConexionOracle`
- `ConexionPostgresql`
- `ConexionSqlServer`
- `Circulo`
- `Cuadrado`
- `Rectangulo`

Todos estos productos concretos pertenecen a la misma naturaleza que su producto base.

Creador

Es la clase que declara el método fábrica.

Responsabilidades:

- Declarar el método que crea productos.
- Trabajar con el producto a través de su interfaz o clase abstracta.
- No depender directamente de un producto concreto.

Ejemplo conceptual:

- `factory_method()`

El método fábrica debe devolver un objeto del tipo producto general.

Creador concreto

Es la subclase que implementa o sobrescribe el método fábrica.

Responsabilidades:

- Decidir qué producto concreto se instancia.
- Devolver un objeto compatible con el tipo producto.
- Encapsular la creación del objeto concreto.

Ejemplo conceptual:

- `CreadorConcreto`

En el material, esta idea aparece con elementos como:

- `Creador_concreto`
- `FactoriaFiguras`

4. Forma práctica de aplicarlo

Pasos:

1. Identificar qué objetos se están creando directamente desde el código cliente o desde una clase principal.
2. Comprobar si esos objetos pertenecen a una misma familia o naturaleza común.
3. Crear una interfaz o clase abstracta para representar el producto general.
4. Hacer que todos los productos concretos implementen o hereden de ese producto general.
5. Crear una clase creadora que declare el método fábrica.
6. Definir el tipo de retorno del método fábrica como el producto general, no como un producto concreto.
7. Crear subclases creadoras concretas.
8. Sobrescribir el método fábrica en cada creador concreto para devolver el producto adecuado.
9. Sustituir las creaciones directas de objetos por llamadas al método fábrica.
10. Hacer que el cliente trabaje con el producto general, sin conocer la clase concreta creada.

5. Idea clave

La clase principal no debe decidir directamente qué clase concreta crear.

Esa decisión se delega en un método fábrica, normalmente implementado por subclases.

Así, el cliente trabaja con una abstracción y la creación concreta queda encapsulada.

6. Ejemplos del material

Ejemplo de conexiones a bases de datos

En el material aparece un ejemplo con una aplicación que utiliza varias bases de datos.

Como cada base de datos necesita una implementación distinta, se define una estructura común para las conexiones.

Producto general:

- `Conexion`

Productos concretos:

- `ConexionMySQL`
- `ConexionOracle`
- `ConexionPostgresql`
- `ConexionSqlServer`

Creador:

- `Creador`

Creador concreto:

- `Creador_concreto`

La idea es que la aplicación pueda pedir una conexión sin depender directamente de si se está creando una conexión MySQL, Oracle, PostgreSQL o SQL Server.

Ejemplo de figuras

También aparece un ejemplo donde se pueden crear distintos tipos de figuras.

Producto general:

- `Figura`

Productos concretos:

- `Circulo`
- `Cuadrado`
- `Rectangulo`

Creador o factoría:

- `Factoria`

Creador concreto:

- `FactoriaFiguras`

La clase creadora se encarga de decidir qué figura concreta crear, mientras que el cliente puede trabajar con el tipo general `Figura`.

Ejemplo de mostrar la hora

En el material también se menciona un ejemplo relacionado con los tipos de mostrar la hora.

Formatos posibles:

- 24 horas
- 12 horas AM/PM

La idea práctica es que ambos formatos son variantes de una misma necesidad: mostrar la hora.

El método fábrica puede encapsular qué tipo concreto de representación se crea o se utiliza.

7. Ejemplo conceptual de uso

En vez de hacer esto directamente en el cliente:

```
new ConexionMySQL()
```

o:

```
new Circulo()
```

El cliente pide el objeto al creador:

```
creador.factoryMethod()
```

El cliente recibe un objeto del tipo general:

- `Conexion`
- `Figura`
- `Producto`

pero no necesita saber exactamente qué clase concreta se ha instanciado.

8. Ventajas prácticas

- Reduce el acoplamiento entre el código cliente y los productos concretos.
- Centraliza la creación de objetos.
- Permite crear objetos concretos en tiempo de ejecución.
- Facilita añadir nuevos productos concretos.
- Permite que las subclases decidan qué objeto crear.
- Evita que el cliente dependa de constructores concretos.
- Separa la lógica de creación de la lógica de uso.
- Ayuda a cumplir el principio de abierto/cerrado, porque se pueden añadir nuevos productos sin modificar el código cliente.

9. Cuidado

Factory Method puede añadir más clases al diseño.

Si solo se crea un objeto concreto y no se espera que existan variantes, puede ser innecesario.

También hay que evitar convertir el método fábrica en un bloque grande de condicionales. Si hay muchas decisiones dentro del método fábrica, puede ser señal de que conviene crear varios creadores concretos.

10. Diferencia práctica con Abstract Factory

Factory Method se centra en delegar la creación de un producto concreto.

Abstract Factory se centra en crear familias completas de productos relacionados.

Por tanto:

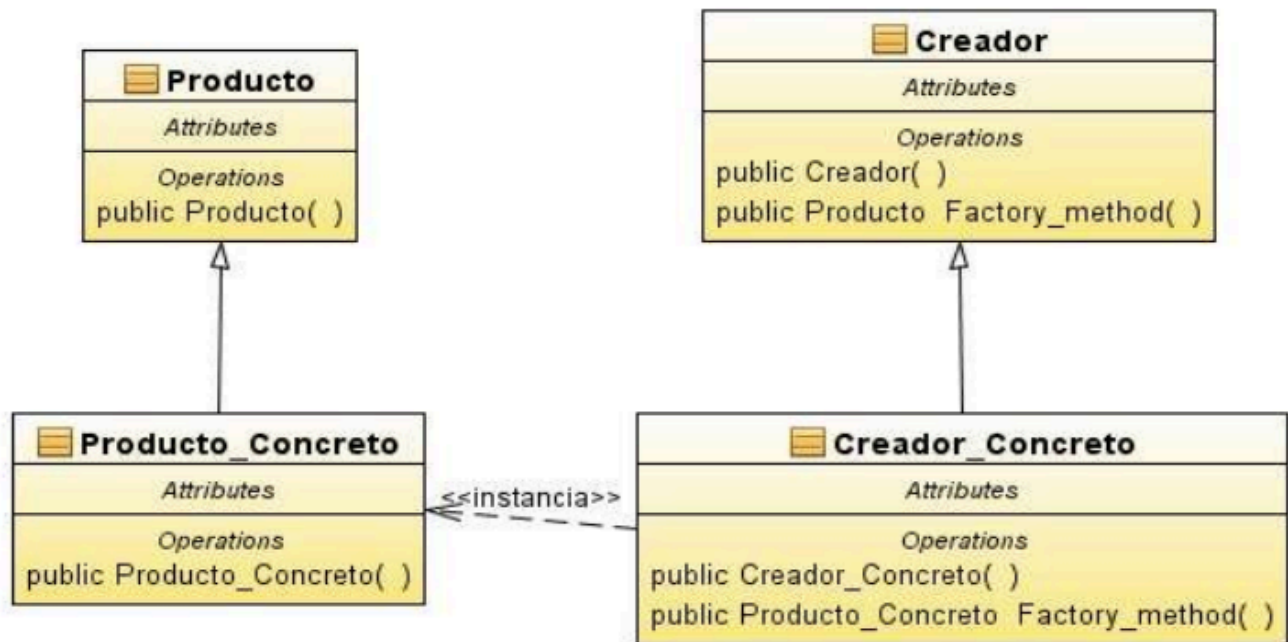
- Factory Method suele trabajar con un tipo principal de producto.
- Abstract Factory trabaja con varios tipos de productos que pertenecen a una misma familia.

Ejemplo:

- Factory Method: crear una `Conexion` concreta.

- Abstract Factory: crear varios productos relacionados de una misma familia, como Automovil y Scooter eléctricos o de gasolina.

11. Plantilla



12. Ejemplo

